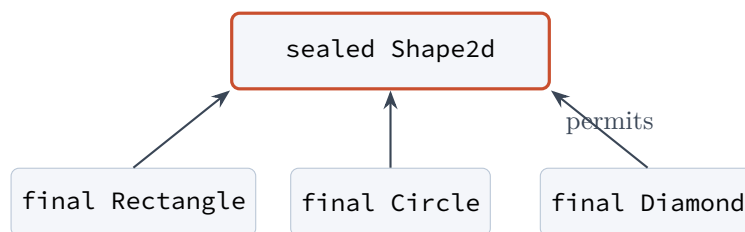


Le interfacce in Java moderno

Contratti, metodi `default` e `static`,
gerarchie `sealed` e `switch` sui tipi



prof. Massimiliano Redolfi

Università degli Studi di Brescia · Ingegneria Informatica

massimiliano.redolfi@unibs.it

Versione 0.1

Quest'opera è distribuita con licenza [Creative Commons](#)
“Attribuzione – Non commerciale – Condividi allo stesso
modo 4.0 Internazionale”.



Indice

3	Le interfacce in Java moderno	1
3.1	Contratti, e dove finiscono	1
3.2	Due modi di ordinare: <code>Comparable</code> e <code>Comparator</code>	2
3.3	Metodi <code>default</code>	3
3.4	Metodi <code>static</code> e <code>private</code>	5
3.5	<code>Comparator</code> , il modello	5
3.6	Un mondo chiuso: <code>sealed</code>	6
3.7	<code>switch</code> sui tipi	7
3.8	Aperto o chiuso?	9
3.9	Riepilogo	9
3.10	Esercizi	10
	Soluzioni	11

Le interfacce in Java moderno

Contratti, classi astratte, Comparable e Comparator, metodi default, static e private, gerarchie sealed e switch sui tipi.

Un'interfaccia, nella sua forma di base, è un contratto: un elenco di metodi che una classe promette di fornire. Per anni è stata solo questo. Poi, in tre riprese, il linguaggio l'ha arricchita: dal 2014 può contenere codice, dal 2017 può dichiarare chiuso l'elenco delle proprie implementazioni, dal 2021 quel mondo chiuso si può esaminare con uno `switch` che il compilatore verifica per completezza. Questa dispensa segue le tre riprese. Prima però rimette a fuoco la forma di base e la confronta con la classe astratta, e usa i due modi di ordinare, `Comparable` e `Comparator`, come esempio guida: sono le interfacce che ogni programma incontra, e mostrano da sole quasi tutto ciò che c'è da dire.

Diamo per note le lambda e le interfacce funzionali, trattate in un'altra dispensa: qui compaiono come strumento, non come argomento.

3.1 Contratti, e dove finiscono

Riprendiamo il bimondo: forme geometriche che hanno un'area e un perimetro.

```
public interface Shape2d {  
    double area();  
    double perimeter();  
}
```

```
public class Rectangle implements Shape2d {  
    private final double width, height;  
  
    public Rectangle(double w, double h) { width = w; height = h; }  
  
    public double area()      { return width * height; }  
    public double perimeter() { return 2 * (width + height); }  
}
```

I metodi di un'interfaccia sono implicitamente `public` e astratti; le implementazioni devono dichiararli `public`. Una classe può implementare più interfacce, e un'interfaccia può estenderne altre: `interface Drawable3d extends Drawable`. Chi usa una `Shape2d` non sa e non deve sapere quale classe ha davanti: è questa ignoranza voluta che permette di aggiungere forme senza toccare il codice che le usa.

3.1.1 Interfaccia o classe astratta?

Una *classe astratta* è una classe che non si può istanziare, pensata per essere estesa, e che può lasciare alcuni metodi senza corpo:

```
public abstract class AbstractShape implements Shape2d {  
    private String name;                // stato condiviso  
  
    protected AbstractShape(String name) { this.name = name; }  
  
    public String name() { return name; } // implementazione condivisa  
  
    public abstract double area();       // da fornire nelle sottoclassi  
}
```

Le due cose si somigliano ma rispondono a domande diverse. La classe astratta serve quando più classi condividono *stato* (campi) e una parte dell'implementazione: è una radice comune, e una classe può averne una sola, perché l'ereditarietà è singola. L'interfaccia serve quando si vuole descrivere una *capacità* che classi anche lontanissime possono avere: una classe ne implementa quante ne vuole. In pratica conviene definire sempre il contratto come interfaccia e, se serve, offrire a parte una classe astratta che lo implementi in modo parziale, come fa la libreria con `List` e `AbstractList`. Vedremo fra poco che i metodi `default` hanno ridotto molto i casi in cui la classe astratta è necessaria.

Osservazione 3.1

Le costanti `static final` sono ammesse in un'interfaccia, ma usarla come contenitore di costanti (senza metodi, per poi “implementarla” e avere i nomi a portata di mano) è un abuso di un'altra epoca: le costanti stanno nella classe che le usa, o in un `enum`.

3.2 Due modi di ordinare: `Comparable` e `Comparator`

Ordinare una lista di forme richiede di sapere quando una forma “viene prima” di un'altra. La libreria offre due contratti per dirlo, ed è istruttivo vedere perché ne servono due.

```
public interface Comparable<T> {  
    int compareTo(T other); // < 0, 0 o > 0: this prima, uguale o dopo other  
}  
  
public interface Comparator<T> {  
    int compare(T a, T b); // stessa convenzione, fra a e b  
}
```

`Comparable` è implementato *dalla classe stessa*: un `Rectangle` che implementa `Comparable<Rectangle>` dichiara il proprio *ordinamento naturale*, uno solo. `Comparator` è un oggetto *esterno* che confronta due elementi: ce ne possono essere quanti si vuole, per area, per perimetro, per nome, e si scelgono al momento dell'ordinamento.

```
public class Rectangle implements Shape2d, Comparable<Rectangle> {
    ...
    public int compareTo(Rectangle other) {
        return Double.compare(area(), other.area());    // per area
    }
}
```

```
List<Rectangle> rs = ...;
Collections.sort(rs);                                // usa compareTo
rs.sort((a, b) -> Double.compare(a.perimeter(), b.perimeter()));
```

Il parametro di tipo `<T>` rende l'interfaccia *generica*: `Comparable<Rectangle>` promette di confrontarsi con rettangoli, e il compilatore rifiuta il confronto con altro. `Double.compare` è il modo corretto di confrontare due `double` restituendo un intero: la sottrazione `a - b` convertita in `int` è un errore classico, perché `0.3 - 0.1` diventa `0`.

Definizione 3.1: Il contratto di un ordinamento

Un confronto deve essere *antisimmetrico* (se $a < b$ allora $b > a$), *transitivo* (se $a < b$ e $b < c$ allora $a < c$) e coerente con se stesso (a confrontato con a dà `0`). È inoltre *consigliato* che sia coerente con `equals`: due elementi uguali dovrebbero confrontarsi a `0`, altrimenti `TreeSet` e `TreeMap`, che usano il confronto e non `equals`, si comportano in modo diverso da `HashSet` e `HashMap`.

Torneremo su `Comparator` nella §3.5: è l'esempio migliore di come i metodi con corpo abbiano cambiato le interfacce.

3.3 Metodi default

Fino a Java 7 un'interfaccia non poteva contenere codice, e questo creava un problema serio a chi manteneva una libreria: aggiungere un metodo a un'interfaccia pubblicata rompeva *tutte* le classi che la implementavano, ovunque nel mondo, perché nessuna forniva il metodo nuovo. Quando in Java 8 si è voluto aggiungere `forEach` a `Iterable` e `sort` a `List`, questo vincolo è stato rimosso.

Definizione 3.2: Metodo default

JAVA 8 Un metodo di un'interfaccia dichiarato `default` ha un corpo. Le classi che implementano l'interfaccia lo ereditano senza doverlo scrivere, e possono ridefinirlo. Il metodo può chiamare gli altri metodi dell'interfaccia, astratti o no.

```
public interface Shape2d {
    double area();
    double perimeter();

    default String describe() {
        return getClass().getSimpleName() + " di area " + area();
    }
}
```

`Rectangle`, `Circle` e `Diamond` ottengono `describe()` senza scrivere una riga; se una di loro vuole una descrizione diversa, la ridefinisce come qualunque altro metodo. Il corpo di `describe` usa `area()`, che nell'interfaccia è astratto: è lecito, perché quando `describe` verrà eseguito ci sarà sempre un oggetto concreto sotto, con la sua `area`. È lo stesso schema del *template method* delle classi astratte, ora disponibile senza ereditarietà.

Tre usi tipici. *Evolvere* un'interfaccia già pubblicata, come nel caso storico. *Offrire comodità* costruite sui metodi astratti, così che ogni implementazione le abbia gratis: `describe`, o un `isEmpty()` definito come `size() == 0`. *Ridurre il lavoro* di chi implementa, lasciando astratto solo il minimo indispensabile.

3.3.1 Chi vince: le regole di risoluzione

Con i metodi default una classe può ereditare lo stesso metodo da più parti. Le regole sono tre, in ordine di priorità.

1. **La classe vince.** Un metodo ereditato da una classe padre (o definito nella classe stessa) ha sempre la precedenza su un default di interfaccia, anche se il default è “più recente”.
2. **L'interfaccia più specifica vince.** Se `B` `extends` `A` e entrambe definiscono `m()` come default, per una classe che implementa `B` vale la versione di `B`.
3. **Altrimenti, la classe deve scegliere.** Se due interfacce senza parentela offrono lo stesso default, il compilatore rifiuta la classe finché non ridefinisce il metodo; nel corpo può richiamare la versione voluta con `Interfaccia.super.metodo()`.

```
interface Drawable { default String describe() { return "disegnabile"; } }
interface Shape2d { default String describe() { return "forma"; } }

class Circle implements Shape2d, Drawable {
    @Override
    public String describe() {
        return Shape2d.super.describe() + ", " + Drawable.super.describe();
    }
}
```

Osservazione 3.2

La prima regola è quella che rende i default sicuri: aggiungere un default a un'interfaccia non può cambiare il comportamento di una classe che già aveva quel metodo. La terza è quella che ogni tanto sorprende in aula, ma è la scelta più prudente: davanti a un'ambiguità il compilatore non tira a indovinare.

3.4 Metodi **static** e **private**

```
public interface Shape2d {
    double area();
    double perimeter();

    static Shape2d square(double side) {           // factory
        return new Rectangle(side, side);
    }

    default String describe() { return name() + " di area " + fmt(area()); }
    default String brief()    { return name(); }

    private String name()      { return getClass().getSimpleName(); }
    private String fmt(double v) { return String.format("%.2f", v); }
}
```

Un metodo **static** di interfaccia **JAVA 8** si chiama sul nome dell'interfaccia, `Shape2d.square(3)`, e *non* viene ereditato dalle implementazioni: è il posto naturale per le *factory* (metodi che costruiscono istanze scegliendo la classe concreta) e per le utilità legate al tipo. Prima di Java 8 queste finivano in classi di appoggio come `Collections` accanto a `Collection`: oggi `List.of(...)` sta in `List`.

Un metodo **private** **JAVA 9** serve a condividere codice fra i **default** senza esporlo: `name()` e `fmt()` sono dettagli di `describe` e `brief`, e nessuna implementazione deve vederli né poterli chiamare.

Membro	Da	Corpo	Ereditato
metodo astratto	sempre	no	sì (da implementare)
costante <code>static final</code>	sempre	—	sì
metodo default	Java 8	sì	sì (ridefinibile)
metodo static	Java 8	sì	no
metodo private	Java 9	sì	no (visibile solo nell'interfaccia)

3.5 **Comparator**, il modello

Mettendo insieme un solo metodo astratto, metodi **static** e metodi **default**, `Comparator` è diventato l'esempio di come si progetta un'interfaccia oggi. Ordinare le forme per area, poi per perimetro a parità di area, in ordine decrescente:

```
shapes.sort(Comparator.comparing(Shape2d::area)           // static
                .thenComparing(Shape2d::perimeter)        // default
                .reversed());                             // default
```

`comparing` è un metodo **static** che riceve una funzione “estrai la chiave” e costruisce il `Comparator` che confronta le chiavi. `thenComparing` e `reversed` sono **default**: ricevono il `Comparator` su cui sono chiamati (`this`) e ne restituiscono uno nuovo. Nessuno di questi metodi è astratto: il solo metodo astratto resta `compare`, e per questo `Comparator` si può ancora scrivere come `lambda`.

```
Comparator<String> perLunghezza = Comparator.comparing(String::length);
Comparator<String> poiAlfabetico =
    perLunghezza.thenComparing(Comparator.naturalOrder());
Comparator<String> nullInFondo = Comparator.nullsLast(perLunghezza);
```

Lo schema, *un metodo astratto più metodi di composizione*, ricorre in tutta la libreria moderna (`Predicate.and`, `Function.andThen`) e vale la pena adottarlo nelle proprie interfacce quando ha senso combinare gli oggetti.

Ricetta: progettare un'interfaccia

1. **Il minimo astratto.** Lasciare astratti solo i metodi che davvero cambiano da un'implementazione all'altra.
2. **Le comodità come default.** Tutto ciò che si può esprimere in termini dei metodi astratti va in un default: le implementazioni lo avranno gratis e potranno ridefinirlo.
3. **Le factory come static.** Costruttori “con un nome” che scelgono la classe concreta, così chi usa l'interfaccia non deve conoscerla.
4. **Il codice comune come private.** Invisibile a tutti tranne che all'interfaccia stessa.
5. **Un solo metodo astratto?** Allora è un'interfaccia funzionale: annotarla `@FunctionalInterface` e accettarla come lambda.

3.6 Un mondo chiuso: **sealed**

Le forme del bimondo sono un insieme fissato: rettangoli, cerchi, rombi. Ma un'interfaccia, per sua natura, è aperta: chiunque, in qualunque file, può scrivere `class Blob implements Shape2d`. Per molte interfacce è esattamente ciò che si vuole (`Drawable`, `Comparable`). Per altre no: quando l'elenco dei casi è parte del significato, come le forme di un gioco, i tipi di un messaggio, i nodi di un'espressione, vorremmo dichiararlo al compilatore e farcelo verificare.

Definizione 3.3: Interfaccia **sealed**

JAVA 17 Un'interfaccia (o una classe) dichiarata **sealed** elenca con `permits` le sole classi e interfacce che possono estenderla direttamente. Ciascuna di esse deve dichiarare come prosegue la gerarchia: **final** (nessuna sottoclasse), **sealed** (a sua volta chiusa) oppure **non-sealed** (riaperta).

```
public sealed interface Shape2d
    permits Rectangle, Circle, Diamond {
    double area();
    double perimeter();
}

public final class Rectangle implements Shape2d { ... }
public final class Circle implements Shape2d { ... }
public final class Diamond implements Shape2d { ... }
```

Se qualcuno scrive `class Blob implements Shape2d`, il compilatore rifiuta: `Blob` non è nell'elenco. Se dimentica `final` su `Circle`, rifiuta anche quello. Il mondo è chiuso, e lo è per costruzione, non per convenzione.

Due dettagli pratici. Le classi permesse devono stare nello stesso package dell'interfaccia (o nello stesso modulo, se il progetto usa i moduli): la chiusura è un fatto locale, non si può chiudere una gerarchia i cui pezzi sono sparsi. E se le classi permesse sono dichiarate nello stesso file dell'interfaccia, la clausola `permits` si può omettere: il compilatore la ricava. È comodo per gerarchie piccole:

```
sealed interface Expr {}

final class Num implements Expr {
    final double value;
    Num(double v) { value = v; }
}
final class Add implements Expr {
    final Expr left, right;
    Add(Expr l, Expr r) { left = l; right = r; }
}
final class Mul implements Expr {
    final Expr left, right;
    Mul(Expr l, Expr r) { left = l; right = r; }
}
```

Un'espressione aritmetica è un numero, una somma di due espressioni o un prodotto di due espressioni: nient'altro. Questa gerarchia è il pane quotidiano di interpreti, compilatori e di ogni programma che elabori dati strutturati. Vediamo ora che cosa il mondo chiuso permette di fare.

3.7 switch sui tipi

Con una gerarchia aperta, distinguere i casi richiede una catena di `instanceof` e un ramo finale per "tutto il resto", che il compilatore non può controllare. Con una gerarchia chiusa lo `switch` può farlo.

Definizione 3.4: Pattern di tipo nello switch

JAVA 21 Un case può contenere un *pattern di tipo*, case `Rectangle r ->`, che vale se l'espressione dello `switch` è un `Rectangle` e in tal caso lo lega alla variabile `r`. Se il tipo dell'espressione è `sealed`, il compilatore verifica che i case coprano tutte le classi permesse, e lo `switch` è *esaustivo* senza default.

```
static String describe(Shape2d s) {
    return switch (s) {
        case Rectangle r -> "rettangolo " + r.width() + "x" + r.height();
        case Circle c -> "cerchio di raggio " + c.radius();
        case Diamond d -> "rombo " + d.d1() + "x" + d.d2();
    }; // senza default: una forma in piu' = errore di compilazione
}
```

Questo è il guadagno del mondo chiuso. Se domani aggiungiamo `Triangle` al `permits`, ogni `switch` su `Shape2d` sparso nel programma smette di compilare finché non gestisce il triangolo: l'elenco dei punti da aggiornare lo produce il compilatore. Con la catena di `instanceof` quell'elenco andava ricostruito a mano, e il caso dimenticato si scopriva a runtime, o non si scopriva.

L'interprete di espressioni è lo stesso schema, ricorsivo:

```
static double eval(Expr e) {
    return switch (e) {
        case Num n -> n.value;
        case Add a -> eval(a.left) + eval(a.right);
        case Mul m -> eval(m.left) * eval(m.right);
    };
}

eval(new Add(new Num(2), new Mul(new Num(3), new Num(4)))); // 14.0
```

3.7.1 Guardie, ordine, null e default

Un `case` può avere una condizione aggiuntiva, introdotta da `when`:

```
String size = switch (s) {
    case Circle c when c.radius() > 10 -> "cerchio grande";
    case Circle c -> "cerchio";
    case Rectangle r -> "rettangolo";
    case Diamond d -> "rombo";
};
```

I `case` sono provati *in ordine*, e il primo che vale vince: il cerchio grande deve precedere il cerchio generico, altrimenti il secondo lo “dominerebbe” e il compilatore segnalerebbe il caso irraggiungibile. Lo stesso vale per i tipi: un `case` su un supertipo deve venire dopo i `case` sui suoi sottotipi.

Se l'espressione può essere `null`, uno `switch` tradizionale lancia `NullPointerException`; con i pattern si può gestire esplicitamente con `case null ->`. Infine il `default`: resta possibile, ma rinuncia proprio alla verifica di esaustività, perché qualunque caso nuovo ci finisce dentro in silenzio. Con una gerarchia `sealed` conviene non scriverlo.

Attenzione 3.1

Lo `switch` sui tipi è uno strumento per chi *usa* la gerarchia, non per chi la definisce. Se ogni forma sa calcolare la propria area, `area()` è un metodo polimorfico, non uno `switch`. Lo `switch` è la scelta giusta quando l'operazione non appartiene alle forme (stamparle in un formato, contarle, tradurle in un altro modello) o quando le classi non sono nostre e non possiamo aggiungervi metodi.

3.8 Aperto o chiuso?

Le due tecniche, polimorfismo su gerarchia aperta e **switch** su gerarchia chiusa, sono speculari, e la scelta dipende da che cosa ci si aspetta che cambi.

	Gerarchia aperta	Gerarchia sealed
chi aggiunge un tipo	chiunque, ovunque	solo chi possiede l'interfaccia
aggiungere un tipo costa	una classe nuova	una classe nuova, più un case in ogni switch
aggiungere un'operazione costa	un metodo in ogni classe	uno switch in un posto solo
il compilatore controlla	che ogni classe implementi i metodi	che ogni switch copra i tipi
quando	plugin, librerie, estensioni di terzi	domini fissati: forme, comandi, messaggi, espressioni

Se il sistema crescerà per *tipi* (nuove forme, nuovi componenti) conviene la gerarchia aperta con i metodi nelle classi. Se crescerà per *operazioni* (nuovi modi di elaborare le stesse espressioni: valutarle, stamparle, semplificarle) conviene la gerarchia chiusa e uno **switch** per operazione. Nessuna delle due rende gratis l'altra direzione: è un problema noto, e le due tecniche sono le sue due risposte.

Osservazione 3.3

Un **enum** è il caso limite della gerarchia chiusa: un insieme fisso di *istanze*, tutte dello stesso tipo e senza dati propri. **sealed** serve quando i casi hanno *forme diverse*: un **Num** porta un numero, un **Add** porta due sottoespressioni. Fra i due, la scelta è quasi sempre ovvia.

3.9 Riepilogo

- Un'interfaccia è un **contratto**; una classe astratta condivide **stato** e implementazione parziale. Contratto come interfaccia, classe astratta solo se serve.
- **Comparable<T>** è l'**ordine naturale**, uno per classe; **Comparator<T>** è un ordine **esterno**, quanti se ne vogliono. Confrontare i **double** con **Double.compare**.
- I metodi **default** hanno un corpo e si ereditano; risoluzione: la classe vince, poi l'interfaccia più specifica, altrimenti la classe deve ridefinire e può usare **Interfaccia.super.m()**.
- I metodi **static** non si ereditano (factory); i **private** condividono codice fra i **default**.
- **Comparator** è il modello: **un metodo astratto più metodi di composizione**.
- **sealed + permits** chiude la gerarchia; le classi permesse sono **final**, **sealed** o **non-sealed**.
- Lo **switch sui tipi** con **case Tipo t ->** è esaustivo su una gerarchia chiusa; guardie con **when**, ordine dal più specifico al più generale, **default** da evitare.
- Aperto se cresceranno i **tipi**, chiuso se cresceranno le **operazioni**.

3.10 Esercizi

Esercizio 3.1: Ordine naturale

Rendere `Circle` `Comparable<Circle>` per raggio. Poi ordinare una lista di cerchi in tre modi: con l'ordine naturale, per raggio decrescente, e per area con i raggi uguali a parità. Quale dei tre richiede una lambda vera?

Esercizio 3.2: Chi vince?

Per ciascuno dei tre frammenti dire se compila e, se sì, che cosa stampa `new C().m()`.

```
// (1)
interface A { default String m() { return "A"; } }
class B { public String m() { return "B"; } }
class C extends B implements A { }
```

```
// (2)
interface A { default String m() { return "A"; } }
interface B extends A { default String m() { return "B"; } }
class C implements A, B { }
```

```
// (3)
interface A { default String m() { return "A"; } }
interface B { default String m() { return "B"; } }
class C implements A, B { }
```

Esercizio 3.3: Espressioni: stampa e semplificazione

Con la gerarchia `Expr` della §3.6, scrivere `String show(Expr e)` che produca `(2 + (3 * 4))`, e `Expr simplify(Expr e)` che sostituisca `x * 1` con `x` e `x + 0` con `x`, ricorsivamente. Poi aggiungere `Neg` (negazione) alla gerarchia e osservare dove il compilatore chiede interventi.

Esercizio 3.4: Da `instanceof` a `switch`

Il metodo seguente lavora su una gerarchia aperta. Renderla `sealed` e riscrivere il metodo come `switch`. Che cosa succede al ramo finale?

```
static double scale(Shape2d s) {
    if (s instanceof Rectangle r) return r.width() * 2;
    if (s instanceof Circle c)    return c.radius() * 2;
    return 1.0;
}
```

Esercizio 3.5: Una factory con un nome

Aggiungere a `Shape2d` un metodo static `Shape2d parse(String s)` che costruisca una forma da una descrizione come `"rect 3 4"`, `"circle 2"`, `"diamond 3 5"`, e lanci `IllegalArgumentException` altrimenti. Perché questo metodo sta bene nell'interfaccia e non in una delle classi?

Soluzioni

Soluzione 3.1.

```
public final class Circle implements Shape2d, Comparable<Circle> {  
    ...  
    public int compareTo(Circle other) {  
        return Double.compare(radius, other.radius);  
    }  
}
```

```
Collections.sort(circles); // ordine naturale  
circles.sort(Comparator.reverseOrder()); // decrescente  
circles.sort(Comparator.comparing(Circle::area)); // per area
```

Nessuno dei tre richiede una lambda: `reverseOrder` inverte l'ordine naturale, e `comparing` riceve un method reference. Poiché l'area di un cerchio dipende solo dal raggio, “per area con i raggi uguali a parità” coincide con l'ordine naturale: la clausola è ridondante, ed è un buon esempio di come un `thenComparing` vada scritto solo quando la chiave secondaria può davvero distinguere.

Soluzione 3.2. (1) Compila e stampa B: la classe vince sul `default`, regola 1. (2) Compila e stampa B: B è più specifica di A, regola 2; il fatto che C dichiari anche A non cambia nulla. (3) Non compila: A e B non hanno parentela e offrono lo stesso `default`; C deve ridefinire `m()`, per esempio `return A.super.m() + B.super.m();`, e allora stampa AB.

Soluzione 3.3.

```

static String show(Expr e) {
    return switch (e) {
        case Num n -> String.valueOf(n.value);
        case Add a -> "(" + show(a.left) + " + " + show(a.right) + ")";
        case Mul m -> "(" + show(m.left) + " * " + show(m.right) + ")";
    };
}

static Expr simplify(Expr e) {
    return switch (e) {
        case Num n -> n;
        case Add a -> {
            Expr l = simplify(a.left), r = simplify(a.right);
            yield (r instanceof Num n && n.value == 0) ? l : new Add(l, r);
        }
        case Mul m -> {
            Expr l = simplify(m.left), r = simplify(m.right);
            yield (r instanceof Num n && n.value == 1) ? l : new Mul(l, r);
        }
    };
}

```

Aggiungendo final class Neg implements Expr, i tre switch (eval, show, simplify) smettono di compilare, ciascuno con un errore che indica il caso mancante. È il comportamento voluto: l'elenco delle operazioni da aggiornare lo produce il compilatore. Si noti in simplify l'uso di yield per produrre il valore da un blocco e di un pattern instanceof annidato nel case.

Soluzione 3.4. Con l'interfaccia dichiarata sealed e i tre permits:

```

static double scale(Shape2d s) {
    return switch (s) {
        case Rectangle r -> r.width() * 2;
        case Circle c -> c.radius() * 2;
        case Diamond d -> 1.0;
    };
}

```

Il ramo finale return 1.0 nascondeva un caso preciso, il rombo, sotto un “tutto il resto” generico. Nello switch esaustivo quel caso va nominato: scrivendolo ci si accorge che il valore 1.0 era probabilmente una dimenticanza, non una scelta. Un default -> 1.0 avrebbe conservato il comportamento, e con esso il difetto.

Soluzione 3.5.

```
static Shape2d parse(String s) {
    String[] p = s.trim().split("\\s+");
    double[] v = new double[p.length - 1];
    for (int i = 1; i < p.length; i++)
        v[i - 1] = Double.parseDouble(p[i]);
    return switch (p[0]) {
        case "rect"    -> new Rectangle(v[0], v[1]);
        case "circle"  -> new Circle(v[0]);
        case "diamond" -> new Diamond(v[0], v[1]);
        default        -> throw new IllegalArgumentException("forma sconosciuta: " + s);
    };
}
```

Il metodo conosce *tutte* le classi concrete e sceglie fra loro: non appartiene a nessuna di esse, e metterlo in `Rectangle` costringerebbe chi vuole un cerchio a passare dal rettangolo. L'interfaccia è l'unico posto che le vede tutte e non è nessuna di loro. Qui il `default` è legittimo: lo `switch` è su una stringa, il cui insieme di valori è aperto per natura.